

HttpOnly cookies and XSRF

Cookie auth · antiforgery · X-XSRF-TOKEN · withCredentials · double-submit

TMS secure mutations

```
withXsrfConfiguration({  
  cookieName: "XSRF-TOKEN",  
  headerName: "X-XSRF-TOKEN",  
})  
req.clone({ withCredentials: true })
```

Secure cookies, antiforgery, and honest Angular wiring

Session outcomes

By the end of this session, you will:

- Implement HttpOnly cookie authentication (.NET ↔ Angular)
- Apply the synchronizer token pattern (XSRF) for unsafe HTTP methods
- Configure ASP.NET Core 10 Antiforgery middleware
- Map the X-XSRF-TOKEN header in Angular HttpClient
- Secure the dashboard against XSS (Cross-Site Scripting) basics

Double-submit strategy

- HttpOnly auth cookie: page script cannot read it shrinks XSS blast radius for the session credential
- XSRF-TOKEN cookie (readable): Angular copies into X-XSRF-TOKEN on unsafe verbs
- Server: antiforgery validates header vs cookie so random sites cannot forge POSTs from the user browser

Flow in prose

Login → .NET sets Auth (HttpOnly) + XSRF-TOKEN (readable) → Angular sends X-XSRF-TOKEN on POST /api/courses → server checks cookie matches header. This pairs with CSP and escaping; it does not erase XSS.

The stolen-token story

- Audit finds TMS prototype storing JWTs in localStorage; a malicious extension reads storage and exfiltrates tokens
- With localStorage: attacker replays JWT against /api/enrollments full access
- HttpOnly session cookie: extension cannot read it; document.cookie omits HttpOnly values
- Trade-off: browsers auto-send cookies add XSRF so attacker pages cannot forge state-changing requests

.NET 10 antiforgery

Program.cs (trimmed)

```
// Program.cs issue XSRF-TOKEN cookie
builder.Services.AddAntiforgery(

    opt => opt.HeaderName = "X-XSRF-TOKEN");

app.Use((ctx, next) => {
    var af = ctx.RequestServices.GetRequiredService<IAntiforgery>();

    var tokens = af.GetAndStoreTokens(ctx);

    ctx.Response.Cookies.Append("XSRF-TOKEN", tokens.RequestToken!,

        new CookieOptions {
            HttpOnly = false, Secure = true,

            SameSite = SameSiteMode.Strict });
    return next(ctx);
});
```

- HttpOnly = false on XSRF-TOKEN only Angular must read it to mirror into X-XSRF-TOKEN; auth cookie stays HttpOnly

Angular HttpClient XSRF + credentials

app.config.ts (trimmed)

```
// app.config.ts providers excerpt
provideHttpClient(
  withFetch(),

  withInterceptors([credentialsInterceptor, errorInterceptor]),

  withXsrfConfiguration({
    cookieName: "XSRF-TOKEN",
    headerName: "X-XSRF-TOKEN",
  }),
),
```

- withXsrfConfiguration names the cookie/header pair Angular mirrors
- credentialsInterceptor (or per-request withCredentials) still required so HttpOnly session cookies are sent separate from XSRF

Credentials interceptor

credentialsInterceptor

```
import { HttpInterceptorFn } from "@angular/common/http";

export const credentialsInterceptor: HttpInterceptorFn =
  (req, next) => next(req.clone({ withCredentials: true }));
```

- Without withCredentials, authenticated cookie sessions look like random 401s after login

Route guards are UX, not security

- A student types `/instructor/dashboard` directly the page loads, fetches data, and the API rejects with 403 Forbidden
- Route Guard (`CanActivateFn`): deflects the user in the browser before the component renders
- Crucial rule: route guards are User Experience, NOT security JS on the user's computer can always be bypassed
- True authorization lives on the .NET API `[Authorize(Roles = "Instructor")]` on the controller action

Junior developers often think hiding an HTML button or creating a Route Guard means a feature is secure. JavaScript running on the user's computer can ALWAYS be bypassed. True authorization is server-side.

Implementing functional route guards

instructorGuard (trimmed)

```
// app/guards/instructor.guard.ts

export const instructorGuard: CanActivateFn = (route, state) => {

  const auth = inject(AuthService);

  const router = inject(Router);

  if (auth.hasRole('Instructor')) {

    return true; // gateway open

  }

  return router.createUrlTree(['/unauthorized']);

};
```

app.routes.ts

```
// app.routes.ts  attach guard

{

  path: 'instructor/dashboard',

  component: InstructorDashboardComponent,

  canActivate: [instructorGuard]

}
```

- canActivate receives an array of functions any guard returning a UrlTree redirects the navigation cleanly
- Pair this with a server-side [Authorize(Roles = "Instructor")] on the controller the guard alone is not enough

When the session ends 401 and 403

errorInterceptor

```
// errorInterceptor 401 branch (trimmed)
export const errorInterceptor: HttpInterceptorFn = (req, next) => {

  const router = inject(Router);
  const snack = inject(MatSnackBar);

  return next(req).pipe(
    catchError((err: HttpResponse) => {

      if (err.status === 401) {

        snack.open("Session Expired", "Login");

        router.navigate(["/login"]);
      }
      throw err;
    })
  );
};
```

- 401: route to login (or refresh) with clear messaging blank lists teach nothing
- 403: authenticated but not allowed different UX than 401; show forbidden/help, not "session expired"
- Pair with M10-Lab-Workbook Exercise 2 and richer ProblemDetails in Exercise 4

Common mistakes Session 2

Mistake	Symptom	Fix
Missing withCredentials: true	401 after successful login	Add credentialsInterceptor or set withCredentials per request
XSRF cookie name mismatch (Angular vs .NET)	400 Antiforgery on POST/PUT/DELETE	Align cookieName with antiforgery Cookie.Name (.NET options)
HttpOnly = true on XSRF cookie	Angular cannot read the cookie	HttpOnly = false on XSRF cookie only
Secure = true on HTTP localhost	Cookie rejected by browser	Make Secure conditional on environment
JWT in JSON body and cookie	Confusion about authoritative session	Cookie for auth; JSON for non-secret profile fields only
Circular dependency (Guard ↔ Service ↔ Router)	Runtime crash or infinite loop	Keep guards small; extract token/role logic into a standalone utility function

Session 2 summary

- HttpOnly cookies keep the primary session credential out of page script reach
- CSRF is mitigated with the double-submit / synchronizer token pattern
- ASP.NET Core 10 Antiforgery is the server-side source of truth for tokens
- Angular HttpClient mirrors XSRF headers when withXsrfConfiguration matches the API
- Secure and SameSite=Strict 2026-style baseline flags for production cookies

Interview angle

Compare localStorage bearer vs HttpOnly cookie sessions: XSS reading storage, withCredentials, and why XSRF matters for cookie-backed POST/PUT/PATCH/DELETE.

Next: Session 3 SignalR and store integration

Hubs, Angular client, automatic reconnection grades and enrollments without manual reload.